Extended reading — Week 02: Bellman Equations and Dynamic Programming

May 21, 2026

How to use this list

Read these five in order. Bellman (1957) and Howard (1960) are the founding documents; Puterman (1994) is the canonical modern textbook chapter you should re-read every couple of years; Tsitsiklis & Van Roy (1997) explains *why* naive DP-with-function-approximation diverges and motivates the contraction-preserving variants you will meet in Week 5 onward; Munos & Szepesvári (2008) bridges DP to learning by delivering the first proper finite-sample bounds for approximate value iteration.

1 Bellman (1957) — Dynamic Programming, Chapters 1–3

Princeton University Press. Republished by Dover, 2003.

Summary. Bellman defines the *principle of optimality* ("an optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision") and derives the recursion that bears his name. The book is short, conversational, and oddly readable for something written in the mid-1950s.

Why it matters for this week. The principle of optimality is the *semantic* content of the Bellman optimality equation. Knowing the equation is one thing; knowing it is a statement about the recursive structure of optimal substructures is what lets you recognise DP when it shows up in disguise (the LP formulation, the linear-system formulation in Eq. 3.8 of the notes, the policy-gradient theorem).

> *"In place of determining the optimal sequence of decisions from the fixed state of the system, we wish to determine the optimal decision to be made at any state of the system. Only if we know the latter do we understand the intrinsic structure of the solution."*

The shift from "sequence of decisions" to "decision per state" is the move from open-loop control to closed-loop policy — the conceptual leap that makes RL possible.

2 Howard (1960) — Dynamic Programming and Markov Processes

MIT Press / Wiley. Chapters 4–6.

Summary. Howard introduces policy iteration (the algorithm that bears his name) and proves it converges in finitely many steps for finite MDPs. The proof uses the policy-improvement theorem of §3.4 of your notes; the finiteness comes from the strict improvement at every step combined with the finite number of deterministic stationary policies.

Why it matters for this week. Howard's PI is the algorithm with *strongest convergence properties* in DP: a guaranteed finite number of sweeps to the exact optimum, no ϵ tolerance. Read it for the cleanest example of a structural-induction proof in optimisation.

> *"A policy that is unimproved by policy improvement is optimal... Since there are only finitely many policies, the algorithm must terminate in finitely many iterations."*

The proof is essentially three sentences. Howard packs the argument into a chapter of motivation, but the core idea is this: each iteration strictly improves, and there are only $|\mathcal{A}|^{|\mathcal{S}|}$ policies, so termination is forced.

3 Puterman (1994) — Markov Decision Processes, Chapters 6–7

Wiley.

Summary. The definitive modern treatment of finite-MDP DP. Chapter 6 contains the proof of contraction (your Theorem 3.2), the existence and uniqueness of $V^\star$, and a polynomial-time bound for policy iteration. Chapter 7 covers modified policy iteration with the proof that any $k \geq 1$ inner sweeps suffice for convergence, plus action elimination (a precursor to Q-learning's targeted updates).

Why it matters for this week. The textbook proofs in Puterman are the cleanest you will find, and they recur as templates in every subsequent DP and ADP analysis. Memorise the inequality $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ from §6.2; you will use it constantly.

> *"Modified policy iteration... unifies value iteration ($k = 1$) and policy iteration ($k = \infty$) as endpoints of a single family of algorithms. The convergence rate is geometric for every k, with constant depending on k and γ."*

Treating VI and PI as endpoints of a single family is the right mental model. When we move to learning, "how many off-policy updates per policy improvement step?" is the same question.

4 Tsitsiklis & Van Roy (1997) — An analysis of temporal-difference learning with function approximation

IEEE TAC 42(5), 674–690. Available also as MIT LIDS-P-2390.

Summary. Proves that TD(0) with *linear* function approximation and on-policy sampling converges to the projection of V^π onto the linear subspace, in a weighted sup-norm. Off-policy, with linear FA, the same algorithm can *diverge* to infinity — the famous Baird counterexample is the canonical demonstration. The paper is the reason "deadly triad" is a phrase in RL.

Why it matters for this week. It is the bridge from exact DP (this week) to approximate DP (Weeks 5, 6, 7, 10). Knowing that *contraction is preserved on-policy but not off-policy under linear FA* explains why DQN needed target networks and Polyak averaging; why TD3 needed twin critics; why Soft Actor-Critic uses entropy regularisation to keep its critic in a contraction basin.

> *"The combination of function approximation, off-policy training, and bootstrapping — the* deadly triad *— is the source of TD divergence. On-policy linear TD(0) converges; remove any one of the three and divergence is precluded; have all three and divergence is generically possible."*

The deadly-triad lens is invaluable: when a deep-RL training run diverges, the first diagnostic is to identify which of the three is at play and whether the algorithm is supposed to handle it.

5 Munos & Szepesvári (2008) — Finite-time bounds for fitted value iteration

JMLR 9, 815–857.

Summary. The first finite-sample analysis of *approximate* value iteration with general function approximation. Given a regression oracle and an exploration distribution μ, the value-iteration error after K iterations is bounded by a sum of (i) the projection error of $T_\star V_K$ onto the function class, (ii) the concentrability coefficient C_μ that measures how well μ covers the on-policy distribution, and (iii) a γ^K term that decays geometrically. The paper invented the modern ADP error decomposition you will see in offline-RL papers.

Why it matters for this week. It tells you what bills come due when you replace the table of V by a neural net: regression error plus a distribution-shift coefficient. The C_μ coefficient is the formal version of the intuition "you cannot evaluate π in regions the data never visits." All of offline RL (Weeks 14, 21) is engineering against C_μ.

> *"The performance loss is bounded by a sum involving the approximation error, the concentrability of the future-state distribution with respect to the sampling distribution, and a γ^K term that vanishes geometrically."*

"Concentrability of the future-state distribution" is the abstract version of "do not extrapolate". It is also the reason behaviour-cloning baselines beat naive offline-RL on out-of-distribution states.

Sourced from

`research/deeprl_core_sota.md` §1 and Sutton & Barto Chapter 4. For modern DP-with-LP formulation see Bertsekas, *Dynamic Programming and Optimal Control* Vol. II (Athena Scientific, 4th ed., 2017).